CodeSport

Sandya Mannarswamy

Welcome to CodeSport! In this column, we provide the solutions to a few of the questions we had featured last month.

Last month's column featured solutions to a medley of questions on computer architecture, operating systems and algorithms. Thanks to our readers M M Sreejith and L Narasimhan for their solutions and inputs. Some readers had expressed doubts regarding Question 4, since the code snippet did not cause any signal on their system, and had requested me to explain the solution in detail. I had warned you that it was a trick question, intended to deceive the reader into thinking that the code ought to cause a SIGBUS—when, in actual fact, the code is bug-free, and will not cause any signal. Consider the relevant part of the code snippet:

```
int* array = (int*) malloc(10);
array++;
*array = 120000;
```

The variable *array* points to an array of integers, and is incremented by the operation *array++*. Consider the general case *array = array + x*, where *x* is an integral value (*x* is 1, in our example). When a pointer variable is added with an integral value, the integral value (namely, 1) is first converted to an address offset, by multiplying it with the size of the base object to which the pointer points. The base object being an integer in this case, the size is *sizeof(integer)*–four bytes. Therefore, the pointer variable is incremented by four bytes when the statement *array++* executes, and so points to the next integer in the array. Hence, dereferencing this pointer as an integer does not cause any SIGBUS signal, since the address is obviously word aligned.

Q So when does a SIGBUS signal get generated? Here is a contrived code snippet to illustrate this:

```
int main(void)
{
  int array[1000];
  int*  p = (int*)((char*)(array) + 1);
  *p = 10;
  printf("%d\n", *p);
}
```

Now, if you compile and execute this code snippet, you would get a SIGBUS. The reason is that here the variable *array*, which points to the first element of the array, has been cast to a pointer to a *char*. When the pointer arithmetic is now performed on this pointer variable, the pointer arithmetic treats the variable as pointing to a char object, and therefore increments the address offset by 1 * *sizeof (char)* (which is equal to 1). For example, if the variable *array* contained the address *1000*, it now becomes *1001*. Therefore, *p* is now assigned to an address which is not word-aligned, and dereferencing it will result in a SIGBUS.

This month's questions

This month, we discuss a couple of graph-related questions. First, we discuss what a bipartite graph is, and the problem of identifying it. Next, we discuss the graph clustering problem.

Bipartite graphs

A bipartite graph is one in which the vertices can be partitioned into two sets, X and Y, such that all edges of the graph are from vertices in X to vertices in Y. If we are asked to colour such a bipartite graph, we will be able to colour the graph with just two colours–with one colour for all vertices in partition X, and one colour for all vertices in partition Y.

Q Now, how do we find out whether a given graph is bipartite or not? Before we answer